

Agentes con LLMs — Arquitecturas y Patrones para Agentes Inteligentes

Taller 2 · Módulo 03: Arquitectura Funcional

Dr. Vicente Machaca Arceda | UTEC

Tabla de Contenidos

1. [Introducción a los Agentes con LLMs](#)
 2. [Componentes de un Agente](#)
 3. [Arquitectura General](#)
 4. [LLM Core: El Cerebro del Agente](#)
 5. [Tools: Las Manos del Agente](#)
 6. [MCP — Model Context Protocol](#)
 7. [Patrones de Diseño de Agentes](#)
 8. [Frameworks y Herramientas](#)
 9. [Observabilidad](#)
 10. [LangSmith](#)
 11. [LangFuse](#)
 12. [LangSmith vs LangFuse: Comparación](#)
 13. [Flujos de Trabajo](#)
 14. [Mejores Prácticas y Métricas](#)
 15. [Glosario Rápido](#)
-

1. Introducción a los Agentes con LLMs

¿Qué es un Agente?

Definición del curso: Sistema autónomo que percibe su entorno, toma decisiones y ejecuta acciones para alcanzar objetivos específicos.

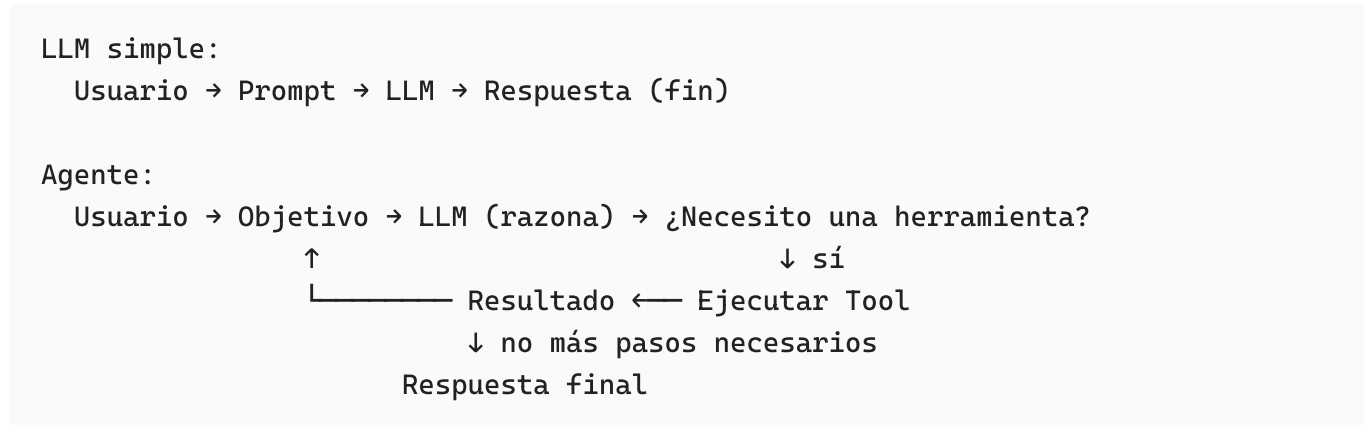
La fórmula que resume todo el taller:

$$\text{Agente} = \text{LLM} + \text{Capacidades Adicionales}$$

Un LLM por sí solo (lo que vimos en el Taller 1) solo **predice texto**. No puede:

- Buscar en internet
- Ejecutar código
- Consultar una base de datos
- Recordar conversaciones pasadas
- Decidir por sí mismo qué hacer a continuación con múltiples pasos

Un **agente** envuelve al LLM con infraestructura que le da estas capacidades. La diferencia clave:



El agente puede **iterar**: pensar, actuar, observar el resultado, pensar de nuevo, hasta resolver la tarea o decidir que terminó.

¿Por qué importan los agentes ahora?

Antes de 2023, usar un LLM significaba: escribir un prompt, recibir una respuesta, fin. Los agentes cambian el paradigma porque permiten que el LLM **actúe en el mundo real** de forma autónoma — reservar un vuelo, depurar código y ejecutar las pruebas, investigar un tema en múltiples fuentes y sintetizar un reporte, etc.

Objetivos de la Lección (según el docente)

Al finalizar deberías poder:

1. **Comprender** la arquitectura de agentes con LLMs
 2. **Identificar** los componentes clave de un agente
 3. **Implementar** patrones arquitectónicos: ReAct, Reflexion, Multi-Agent
 4. **Seleccionar** frameworks apropiados para diferentes casos de uso
 5. **Diseñar** sistemas de agentes robustos y seguros
-

2. Componentes de un Agente

Cuatro capacidades fundamentales distinguen a un agente de un simple chatbot:

Autonomía

Opera sin supervisión constante. Dado un objetivo de alto nivel ("organiza mi viaje a Cusco"), el agente decide los pasos necesarios sin que el humano le diga "ahora busca vuelos", "ahora busca hoteles" en cada paso.

Razonamiento

Planifica y decide estrategias. Esto es el LLM "pensando" sobre qué hacer — descomponer el objetivo en subtarefas, decidir el orden, anticipar qué información necesita.

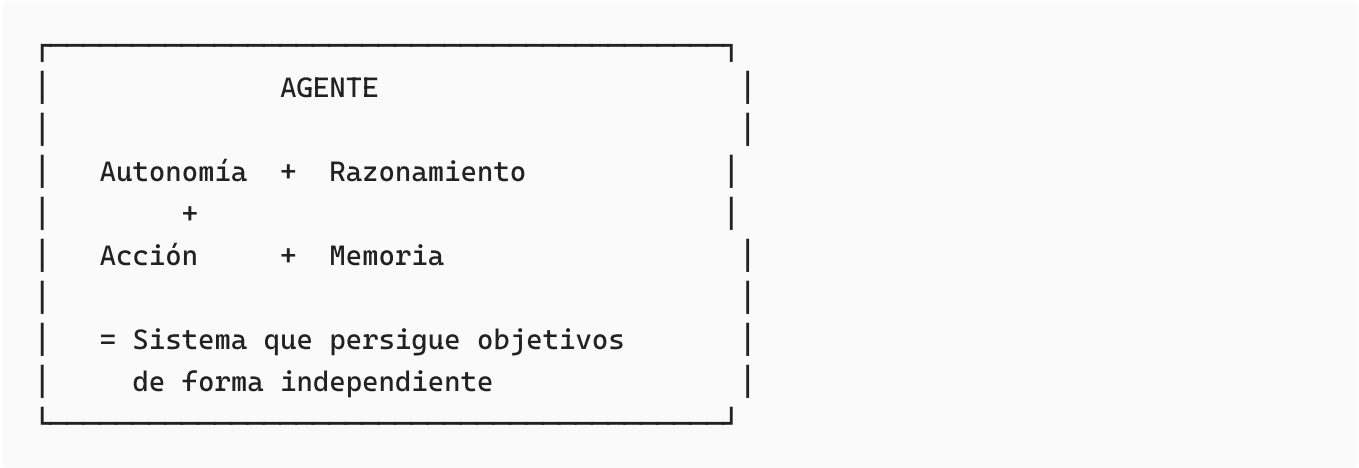
Acción

Interactúa con herramientas y el entorno. Sin esto, el razonamiento es solo teoría — la acción es lo que conecta al LLM con el mundo real (APIs, archivos, bases de datos).

Memoria

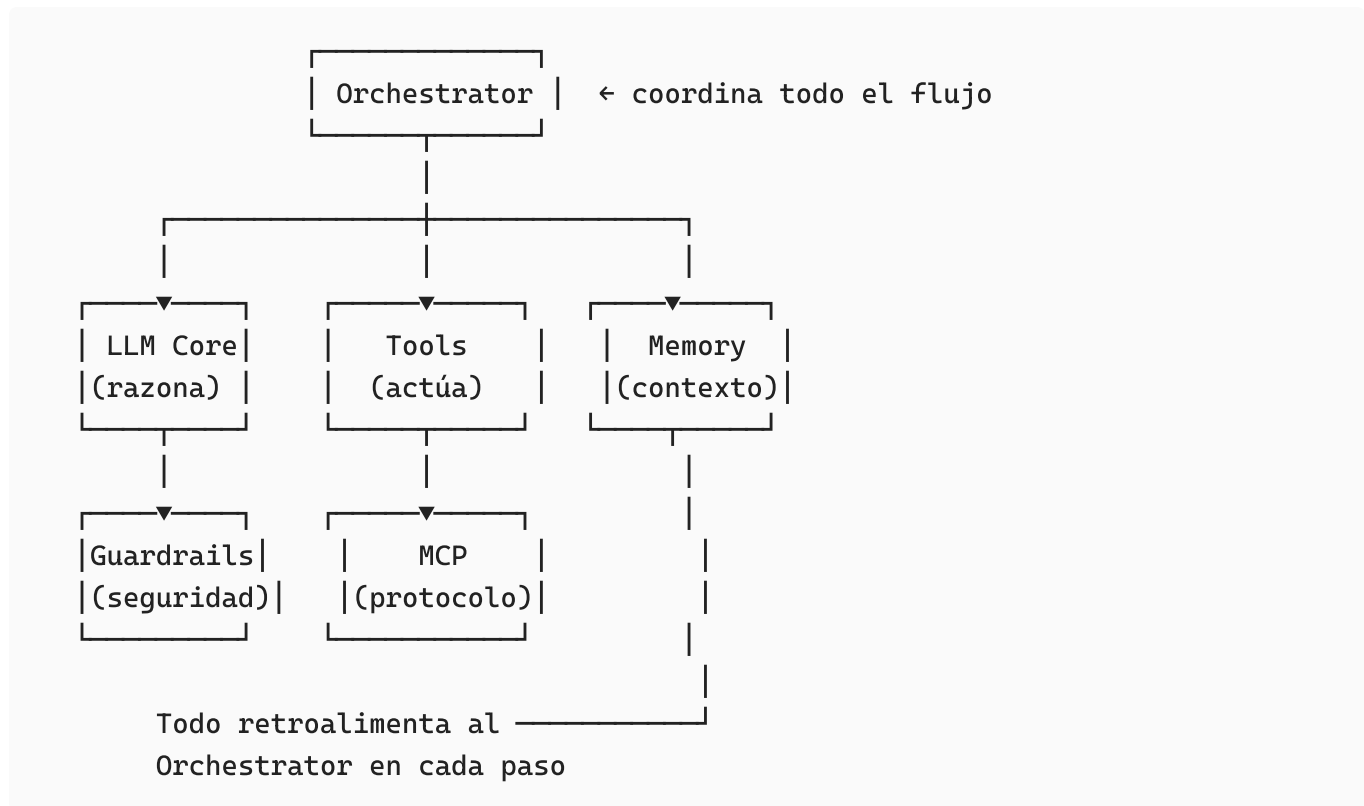
Mantiene contexto y aprende de experiencias. Puede ser:

- **Memoria de corto plazo:** el historial de la conversación actual (limitado por la ventana de contexto — ver Taller 1)
- **Memoria de largo plazo:** información persistida entre sesiones (en una base de datos vectorial, por ejemplo)



3. Arquitectura General

El docente presenta un diagrama con seis bloques que interactúan:



El Orchestrator (orquestador)

Es el "director de orquesta" del agente. Su trabajo:

1. Recibir el objetivo del usuario
2. Decidir si el LLM Core debe razonar más, llamar una herramienta, o responder
3. Enviar la información correcta a cada componente
4. Recopilar resultados y decidir el siguiente paso
5. Determinar cuándo la tarea está completa

En frameworks como LangChain/LangGraph, el orquestador es literalmente el "agent loop" — el bucle de código que repite: *pensar* → *actuar* → *observar* → *repetir*.

Guardrails (barreras de seguridad)

Mecanismos que limitan lo que el agente puede hacer, evitando:

- Ejecutar acciones destructivas sin confirmación (eliminar archivos, hacer compras)
- Filtrar información sensible en las respuestas
- Salirse del alcance previsto (ej. un agente de soporte que empieza a dar consejos médicos)
- Loops infinitos o gasto descontrolado de tokens/dinero

Ejemplos prácticos de guardrails:

```
# Ejemplo conceptual de guardrail simple
def validar_accion(accion, args):
    acciones_peligrosas = ["eliminar_archivo", "transferir_dinero",
"enviar_email_masivo"]
    if accion in acciones_peligrosas:
        return solicitar_confirmacion_humana(accion, args)
    return ejecutar(accion, args)
```

4. LLM Core: El Cerebro del Agente

Componentes del LLM Core

1. Modelo base: GPT-4/5, Claude, Gemini, Llama, etc. — el modelo de lenguaje que hace el razonamiento real (ver Taller 1 para cómo funciona internamente).

2. Prompt Engineering: cómo se estructuran las instrucciones.

- **System prompts:** instrucciones persistentes que definen el rol y comportamiento del agente
- **Templates:** plantillas reutilizables para estructurar inputs consistentemente

3. System Instructions: las reglas explícitas de comportamiento.

```
system_prompt = """
Eres un agente de soporte técnico para una tienda de e-commerce.
Tienes acceso a las siguientes herramientas: buscar_pedido,
procesar_reembolso, escalar_a_humano.

Reglas:
- Nunca proceses un reembolso mayor a $100 sin escalar a un humano.
- Si el cliente está molesto, prioriza empatía antes de resolver el problema
técnico.
- No inventes información sobre políticas que no conoces – usa la herramienta
buscar_politica.
"""
```

4. Parámetros de generación: temperature, top-p, max tokens (revisado en profundidad en el Taller 1, sección de Sampling). Para agentes, generalmente se usa **temperature baja** (0-0.3) porque se necesita consistencia y precisión al decidir qué herramienta llamar — no creatividad.

Consideraciones Clave al Elegir el Modelo

Factor	Pregunta a responder
Capacidades requeridas	¿Necesito razonamiento complejo (modelo grande) o tareas simples (modelo pequeño/rápido)?
Costo	¿Cuántas llamadas hará el agente por tarea? Un agente puede hacer 10-50 llamadas al LLM para una sola tarea compleja
Latencia	¿La aplicación es interactiva (necesita respuesta rápida) o puede ser asíncrona?
Fine-tuning vs Prompting	¿Vale la pena entrenar un modelo especializado, o basta con un buen system prompt + few-shot examples?

Regla práctica: la mayoría de agentes en producción usan un modelo grande/capaz para el razonamiento principal y, opcionalmente, un modelo más pequeño y rápido para sub-tareas simples (clasificación, extracción) — esto se llama **arquitectura de modelos en cascada**.

5. Tools: Las Manos del Agente

Definición

Tools: Funciones o APIs que el agente puede invocar para interactuar con el mundo exterior.

Esto es literalmente la implementación del **Function Calling** que vimos en el Taller 1 — pero ahora desde la perspectiva de "qué herramientas le doy a mi agente" y no solo "cómo funciona técnicamente".

Tipos de Herramientas

Tipo	Ejemplos
APIs externas	Weather API, Search API (Google/Bing), bases de datos SQL/NoSQL
Operaciones de sistema	Leer/escribir archivos (File I/O), ejecutar comandos de shell
Servicios especializados	Enviar emails, crear eventos de Calendar, actualizar un CRM
Herramientas personalizadas	Lógica de negocio específica de tu empresa (ej. "calcular_descuento_cliente_vip")

Anatomía de una Tool bien diseñada

```
def buscar_pedido(numero_pedido: str) -> dict:
    """
    Busca la información de un pedido por su número.

    Args:
        numero_pedido: ID del pedido, formato 'PED-XXXXX'

    Returns:
        dict con estado, productos, fecha_envío
    """
    # ... lógica real ...
    return {"estado": "enviado", "productos": [...], "fecha_envio": "2026-06-20"}
```

Por qué la descripción y los tipos importan tanto: el LLM decide cuándo usar la herramienta basándose únicamente en su nombre, descripción y parámetros. Si la descripción es ambigua, el agente puede:

- No usar la herramienta cuando debería
- Usarla con argumentos incorrectos
- Confundirla con otra herramienta similar

Buenas prácticas para definir tools:

1. Nombres descriptivos y específicos (buscar_pedido_por_id mejor que buscar)
2. Descripciones que digan exactamente cuándo usar la herramienta y cuándo no
3. Validar los argumentos antes de ejecutar (el LLM puede generar argumentos inválidos)
4. Manejar errores con mensajes que el LLM pueda entender y corregir
5. Mantener las herramientas **atómicas** (una función = una responsabilidad clara)

Ejemplo Práctico: Simple Agent

El PDF hace referencia a un ejercicio de "Simple Agent Example". Una implementación mínima con la API de Claude se vería así:

```
import anthropic

client = anthropic.Anthropic()

tools = [{
    "name": "get_weather",
```

```

    "description": "Obtiene el clima actual de una ciudad específica",
    "input_schema": {
        "type": "object",
        "properties": {
            "city": {"type": "string", "description": "Nombre de la ciudad"}
        },
        "required": ["city"]
    }
}
}]

```

```

def get_weather(city: str) -> str:
    # Llamada real a una API de clima
    return f"En {city} hace 22°C y está soleado."

```

```

messages = [{"role": "user", "content": "¿Necesito sombrilla en Lima hoy?"}]

```

```

response = client.messages.create(
    model="claude-opus-4-8",
    max_tokens=1024,
    tools=tools,
    messages=messages
)

```

```

# Bucle del agente: revisar si el modelo quiere usar una herramienta
while response.stop_reason == "tool_use":
    tool_use = next(b for b in response.content if b.type == "tool_use")

```

```

    if tool_use.name == "get_weather":
        result = get_weather(tool_use.input["city"])

```

```

    messages.append({"role": "assistant", "content": response.content})
    messages.append({
        "role": "user",
        "content": [{"type": "tool_result", "tool_use_id": tool_use.id,
"content": result}]
    })

```

```

    response = client.messages.create(
        model="claude-opus-4-8", max_tokens=1024, tools=tools,
messages=messages
    )

```

```

print(response.content[0].text)

```

```

# "Sí, parece que hace calor y sol en Lima, no necesitas sombrilla, pero sí
protector solar."

```


Este bucle `while response.stop_reason == "tool_use"` **es** el corazón de todo agente: el LLM decide, el código ejecuta, el resultado vuelve al LLM, y se repite hasta que el LLM decide que ya puede responder directamente.

6. MCP — Model Context Protocol

Definición

MCP: Protocolo estándar para conectar LLMs con fuentes de datos y herramientas externas.

El Problema que Resuelve MCP

Antes de MCP, cada vez que querías conectar un agente a una nueva fuente de datos (GitHub, Slack, una base de datos, el sistema de archivos), tenías que escribir código de integración específico para ese LLM/framework. Si cambiabas de framework, tenías que reescribir todas las integraciones.

MCP estandariza esto: define un **protocolo común** para que cualquier LLM/agente pueda hablar con cualquier fuente de datos/herramienta que implemente el protocolo, sin código de integración custom.

```
SIN MCP:
  Agente A (LangChain) ↔ código custom ↔ GitHub API
  Agente B (CrewAI)    ↔ código custom ↔ GitHub API
  Agente C (Claude)    ↔ código custom ↔ GitHub API
  (3 integraciones distintas para la misma fuente)

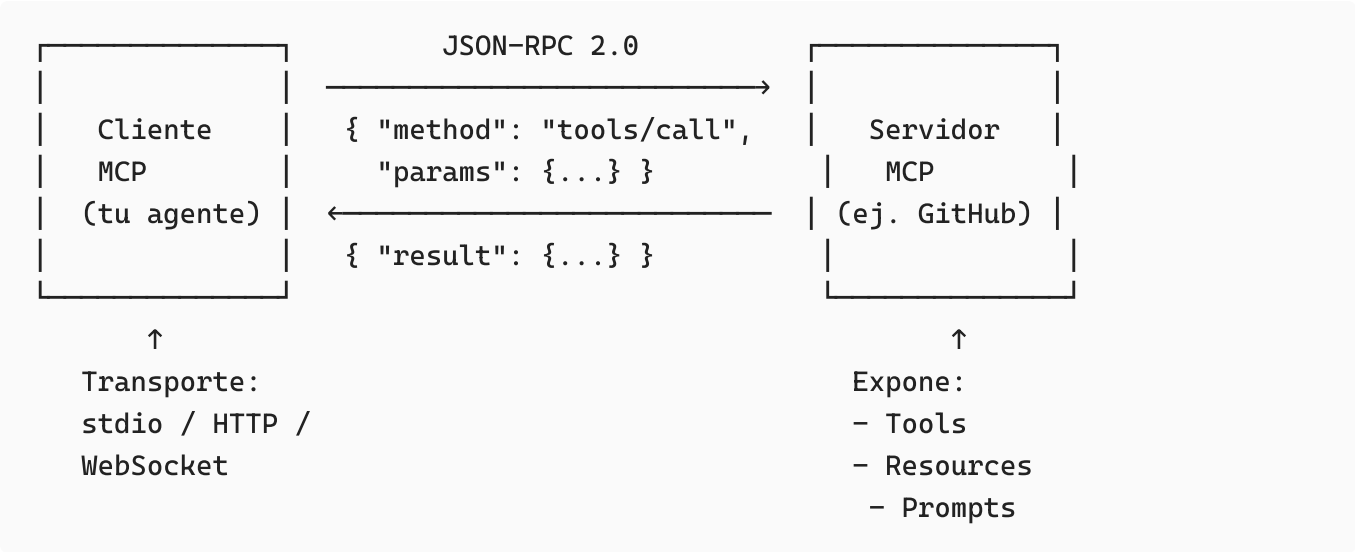
CON MCP:
  Agente A ┌
  Agente B ┤───> Cliente MCP ──> Protocolo MCP ──> Servidor MCP de GitHub
  Agente C └
  (1 servidor MCP, cualquier agente compatible lo usa)
```

Componentes de MCP

Componente	Rol
Servidores MCP	Proveen recursos y herramientas (ej. un servidor MCP para GitHub expone "crear_issue", "leer_archivo", etc.)
Clientes MCP	Consumen los servicios — son los agentes mismos

Componente	Rol
Transporte	Cómo viaja la comunicación: <code>stdio</code> (proceso local), <code>HTTP</code> , <code>WebSocket</code>
Protocolo	El formato de los mensajes: JSON-RPC 2.0

Arquitectura MCP en Detalle



Tres tipos de capacidades que expone un servidor MCP:

1. **Tools**: funciones ejecutables (ej. `create_pull_request`)
2. **Resources**: datos que el LLM puede leer (ej. el contenido de un archivo)
3. **Prompts**: plantillas de prompts reutilizables que el servidor sugiere

Ejemplo de Mensaje JSON-RPC en MCP

```
// Cliente solicita ejecutar una herramienta
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/call",
  "params": {
    "name": "search_repository",
    "arguments": {"query": "bug en login", "repo": "mi-empresa/backend"}
  }
}

// Servidor responde
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
```

```
    "content": [{ "type": "text", "text": "Encontrados 3 issues
relacionados..." }]
  }
}
```

MCP vs Function Calling Tradicional

Aspecto	Function Calling (Taller 1)	MCP
Alcance	Una función definida en tu código, para un LLM específico	Un servidor reusable por cualquier cliente compatible
Reutilización	Tienes que redefinir la función para cada proyecto/framework	El servidor MCP se escribe una vez, se usa en N agentes
Estandarización	Cada proveedor (OpenAI, Anthropic, Google) define su propio formato	JSON-RPC 2.0 estándar, agnóstico al proveedor del LLM
Ecosistema	Tú mantienes todas tus integraciones	Existen servidores MCP públicos ya hechos (Slack, GitHub, filesystem, etc.)

Analogía útil: Function Calling es como escribir el driver de una impresora para un solo programa; MCP es como el estándar USB — cualquier dispositivo compatible funciona con cualquier computadora compatible.

7. Patrones de Diseño de Agentes

Esta es la sección más extensa del taller. El docente presenta 9-10 patrones arquitectónicos, casi todos mostrados solo como diagramas — aquí los explico con profundidad porque son el núcleo conceptual de cómo se diseñan sistemas de agentes reales.

7.1 Sistema de un Solo Agente

```
Usuario → [Agente único: razona + actúa + responde] → Resultado
```

El caso más simple: un agente con sus herramientas resuelve toda la tarea. Útil cuando la tarea es acotada y no requiere especialización (ej. un asistente de soporte simple).

Cuándo usar: tareas bien definidas, bajo riesgo de necesitar "puntos de vista" diferentes, bajo presupuesto de cómputo.

7.2 Sistemas Multiagente Secuenciales

Usuario → Agente 1 → Agente 2 → Agente 3 → Resultado
(investiga) (redacta) (revisa)

Cada agente especializado hace una parte del trabajo y pasa el resultado al siguiente, como una línea de ensamblaje.

Ejemplo real: un pipeline de generación de contenido:

1. Agente investigador busca información sobre el tema
2. Agente escritor redacta el artículo con esa información
3. Agente editor revisa gramática y estilo

Ventaja: cada agente puede tener un prompt/modelo optimizado para su tarea específica.

Desventaja: si un agente falla o produce algo malo, se propaga a los siguientes (no hay paralelismo, es lento).

7.3 Sistemas Multiagente Paralelos

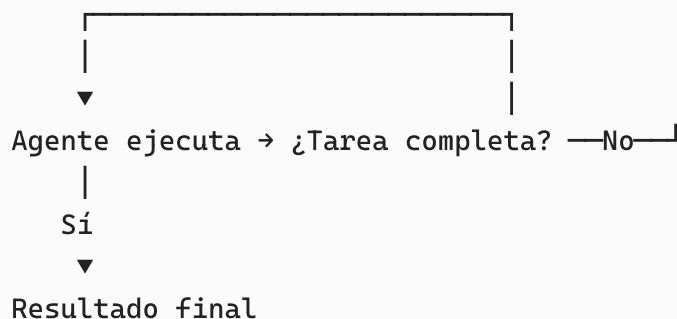
Usuario → Tarea $\begin{cases} \rightarrow \text{Agente 1 (analiza sentimiento)} \\ \rightarrow \text{Agente 2 (extrae entidades)} \\ \rightarrow \text{Agente 3 (clasifica categoría)} \end{cases} \rightarrow \text{Combinador} \rightarrow \text{Resultado}$

Varios agentes trabajan **simultáneamente** sobre la misma entrada (o partes independientes de ella), y luego se combinan los resultados.

Cuándo usar: cuando las subtareas son independientes entre sí. Por ejemplo, analizar una reseña de producto desde tres ángulos distintos al mismo tiempo, en vez de uno tras otro.

Ventaja: mucho más rápido que el secuencial (latencia = la del agente más lento, no la suma de todos).

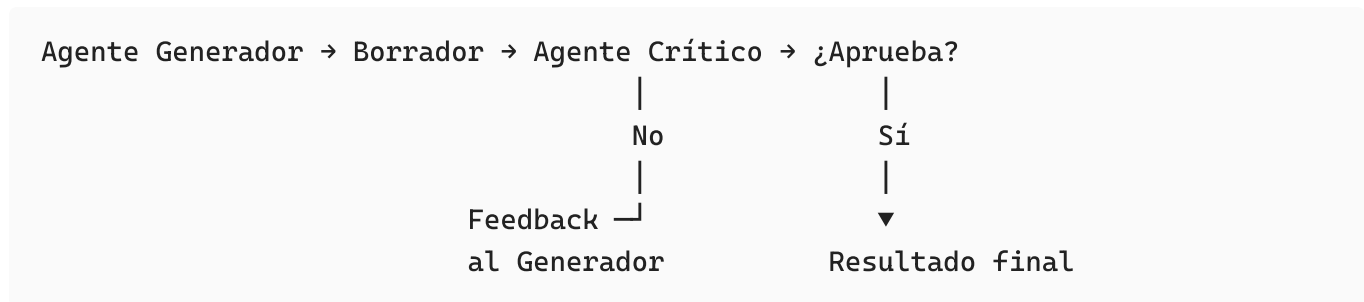
7.4 Sistemas Multiagente en Bucle (Loop)



El agente (o conjunto de agentes) repite un ciclo hasta que se cumple una condición de salida. Es la base de los agentes "autónomos" que trabajan en tareas largas sin supervisión paso a paso.

Riesgo principal: loops infinitos si la condición de salida está mal definida, o gasto descontrolado de tokens. Siempre se necesita un **límite máximo de iteraciones** como guardrail.

7.5 Patrón de Revisión y Crítica

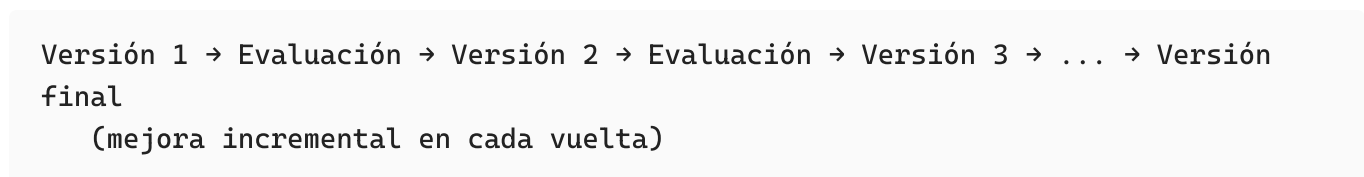


Un agente genera contenido/solución, y un **segundo agente especializado en criticar** evalúa si cumple los criterios de calidad. Si no, da feedback específico y el generador reintenta.

Este patrón es la base conceptual de lo que en la literatura se llama **Reflexion** (uno de los objetivos explícitos del taller) — el agente reflexiona sobre su propio output (o el de otro agente) antes de darlo como final.

Ejemplo: un agente que escribe código, y otro que lo revisa buscando bugs antes de aceptarlo, similar a un code review.

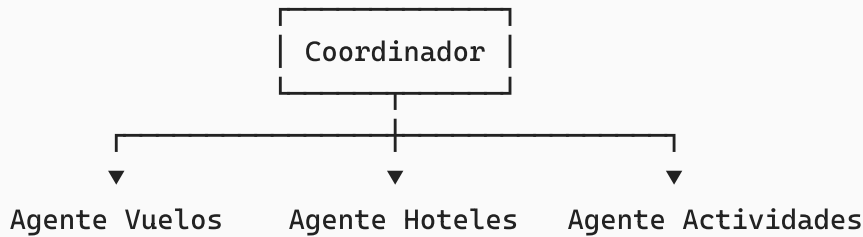
7.6 Patrón de Refinamiento Iterativo



Similar al patrón de revisión y crítica, pero aquí el énfasis está en **múltiples rondas de mejora progresiva**, no solo una corrección. Cada iteración parte del resultado anterior y lo perfecciona.

Diferencia con "Revisión y Crítica": en revisión y crítica hay dos roles fijos (generador/crítico); en refinamiento iterativo puede ser el mismo agente mejorando su propio trabajo repetidamente, con o sin un criterio externo.

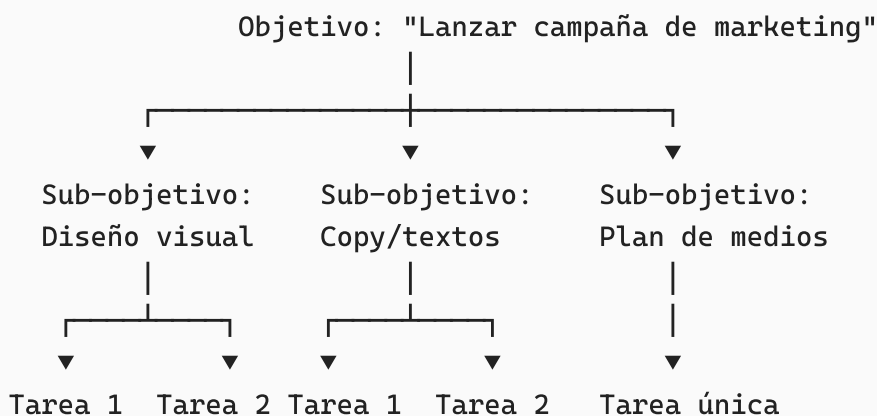
7.7 Patrón de Coordinador



Un agente "coordinador" (a veces llamado "manager" o "supervisor") decide qué sub-agentes especializados invocar y en qué orden, y sintetiza sus respuestas. El coordinador **no resuelve la tarea directamente** — su trabajo es delegar y coordinar.

Diferencia clave vs el patrón secuencial: en secuencial, el flujo está fijo de antemano (siempre 1→2→3). En el patrón de coordinador, el **coordinador decide dinámicamente** a quién llamar según la situación — es más flexible e inteligente.

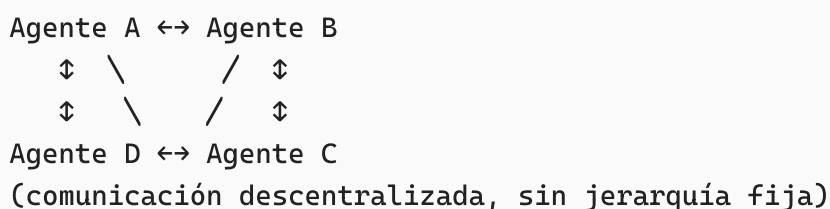
7.8 Patrón de Descomposición Jerárquica de Tareas



Un objetivo complejo se descompone en sub-objetivos, y estos a su vez en tareas atómicas ejecutables — como un árbol. Un agente "planificador" en la raíz genera el árbol; agentes hoja ejecutan las tareas concretas.

Cuándo usar: proyectos complejos con muchas partes interdependientes donde la planificación es tan importante como la ejecución (ej. generar un plan de negocio completo).

7.9 Patrón de Enjambre (Swarm)



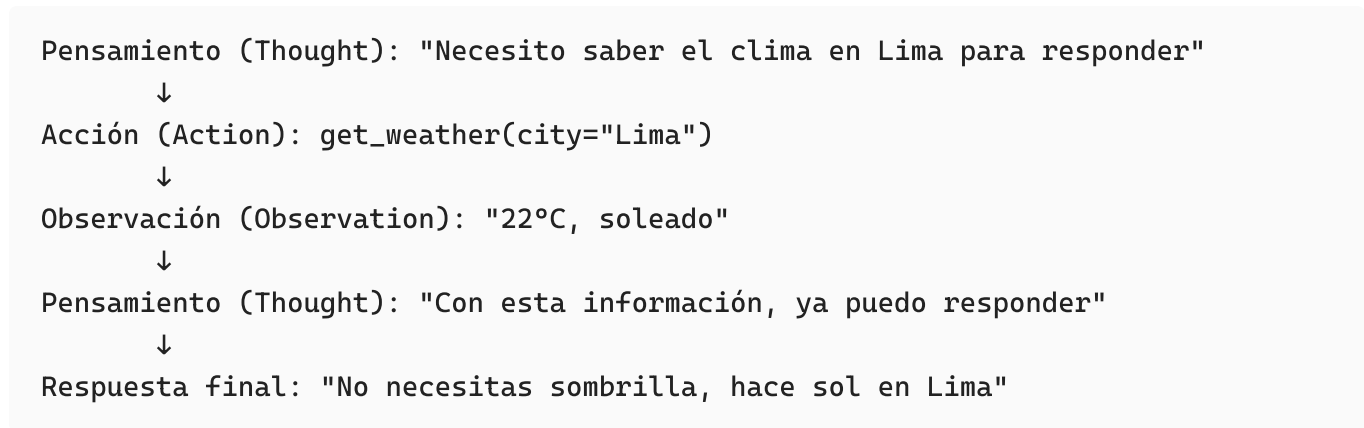
A diferencia del patrón de coordinador (jerárquico, centralizado), en un **enjambre** los agentes se comunican entre sí de forma descentralizada, sin un líder fijo — inspirado en el comportamiento de enjambres en la naturaleza (hormigas, abejas).

Ventaja: muy resiliente — si un agente falla, los demás pueden seguir funcionando. Bueno para problemas donde la solución emerge de la interacción entre muchos agentes simples.

Desventaja: más difícil de predecir y depurar que un sistema jerárquico.

7.10 Patrón ReAct (Reasoning + Acting)

Este es uno de los patrones más importantes y citados en la industria (papel original: Yao et al., 2022).

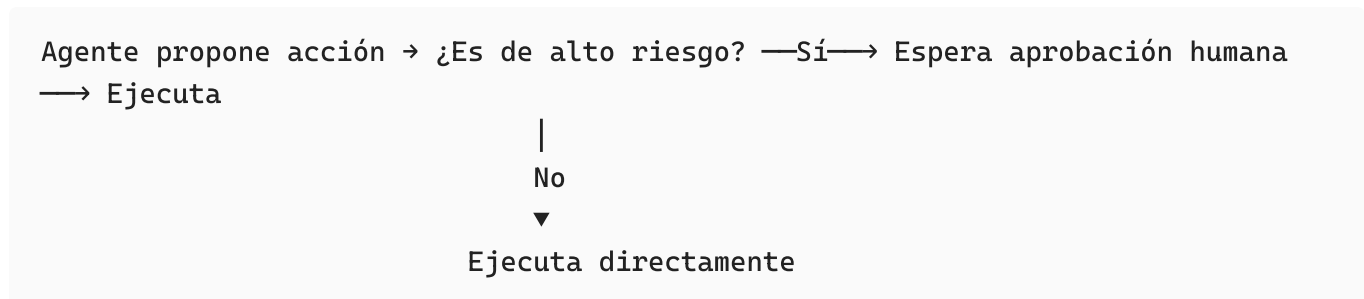


El ciclo ReAct se repite: Thought → Action → Observation → Thought → ... hasta que el modelo decide que tiene suficiente información para responder.

Por qué es poderoso: combina explícitamente el razonamiento (chain-of-thought, visto en el Taller 1) con la capacidad de actuar. El modelo "piensa en voz alta" antes de cada acción, lo que mejora notablemente la precisión de las decisiones y hace el proceso **interpretable** — puedes leer el razonamiento del agente y entender por qué hizo lo que hizo.

Esto es literalmente lo que implementamos en el código del Simple Agent Example (sección 5) — aunque ahí no se muestra el "Thought" explícito en texto, internamente el LLM está razonando antes de decidir llamar a `get_weather`.

7.11 Patrón con Intervención Humana (Human-in-the-Loop)



El agente se detiene antes de ciertas acciones críticas (eliminar datos, gastar dinero, enviar comunicaciones externas) y espera confirmación de un humano.

Esencial para producción: ningún sistema de agentes serio en un dominio de alto riesgo (finanzas, salud, legal) debería operar sin este patrón en las acciones irreversibles. Es la implementación concreta de los **Guardrails** mencionados en la arquitectura general.

7.12 Patrón de Lógica Personalizada

Agente → [Bloque de código determinista, NO un LLM] → Agente continúa

No todo en un sistema de agentes necesita ser "inteligente". A veces la mejor solución es código tradicional: validaciones, cálculos exactos, reglas de negocio fijas. Este patrón reconoce que **mezclar LLMs con código determinista** donde corresponda es más confiable y barato que forzar todo a pasar por el LLM.

Ejemplo: en vez de pedirle al LLM que calcule un descuento con una fórmula compleja (el Taller 1 mostró que los LLMs fallan en aritmética precisa), se usa una función Python determinista para el cálculo, y el LLM solo decide *cuándo* aplicarla.

Resumen Visual de Todos los Patrones

PATRONES DE FLUJO
Secuencial Paralelo En Bucle
PATRONES DE CALIDAD
Revisión y Crítica Refinamiento Iterativo
PATRONES DE ORGANIZACIÓN MULTI-AGENTE
Coordinador Descomposición Jerárquica Enjambre
PATRONES DE RAZONAMIENTO
ReAct
PATRONES DE SEGURIDAD Y CONTROL
Intervención Humana Lógica Personalizada

Nota práctica: estos patrones **no son mutuamente excluyentes**. Un sistema real de producción típicamente combina varios: por ejemplo, un patrón de Coordinador que delega a sub-agentes que usan ReAct internamente, con un patrón de Intervención Humana para las acciones críticas, y Lógica Personalizada para los cálculos exactos.

8. Frameworks y Herramientas

Comparación de Frameworks (según el docente)

Framework	Complejidad	Multi-Agent	Popularidad
LangChain	Alta	Sí	Alta
LangGraph	Alta	Sí	Media
ADK (Google Agent Development Kit)	Baja	Sí	Alta
AutoGen (Microsoft)	Media	Sí	Media
CrewAI	Baja	Sí	Creciente
Semantic Kernel (Microsoft)	Media	Sí	Media

Descripción de Cada Framework

LangChain: el framework más establecido y con mayor ecosistema. Ofrece abstracciones para casi todo (chains, agents, memory, retrievers) pero esa flexibilidad implica mayor complejidad y curva de aprendizaje.

LangGraph: construido sobre LangChain, modela los agentes como **grafos de estados** — ideal para flujos complejos con bucles, branches condicionales y control fino sobre cada paso. Es la opción cuando necesitas el patrón "en bucle" o "jerárquico" con mucho control.

ADK (Agent Development Kit): framework de Google, diseñado para ser simple de usar manteniendo soporte multi-agente. Buena opción para empezar rápido sin sacrificar capacidad de escalar a sistemas multi-agente.

AutoGen: framework de Microsoft Research enfocado en **conversaciones entre múltiples agentes** — varios agentes "chatean" entre sí para resolver una tarea, similar al patrón de enjambre o coordinador.

CrewAI: diseñado específicamente alrededor del concepto de "roles" — defines un equipo (crew) de agentes con roles claros (ej. "investigador", "escritor", "editor") similar al patrón secuencial o de coordinador, con una API muy simple.

Semantic Kernel: framework de Microsoft orientado a integrar LLMs en aplicaciones .NET/Python empresariales existentes, con fuerte énfasis en "plugins" (equivalente a tools).

¿Cómo elegir?

Si necesitas...	Considera
Máxima flexibilidad y ecosistema maduro	LangChain
Control fino sobre flujos complejos con estados	LangGraph
Empezar rápido, simple, pero escalable	ADK o CrewAI
Conversaciones ricas entre múltiples agentes	AutoGen
Integración en stack empresarial Microsoft/.NET	Semantic Kernel

9. Observabilidad

El Desafío de las Aplicaciones LLM

Las aplicaciones basadas en LLMs (y más aún, agentes) presentan problemas que el desarrollo de software tradicional no tiene de la misma forma:

Problemas comunes:

- **Respuestas inesperadas:** el mismo prompt puede dar resultados distintos
- **Latencia variable:** depende del proveedor, la carga, el tamaño del contexto
- **Costos impredecibles:** un agente puede hacer 3 o 30 llamadas al LLM para la misma tarea, según cómo razone
- **Errores difíciles de rastrear:** ¿el bug está en el prompt, en una tool, en cómo se parseó la respuesta?
- **Rendimiento no medible:** sin instrumentación, no sabes si tu agente "funciona bien" más allá de probarlo manualmente

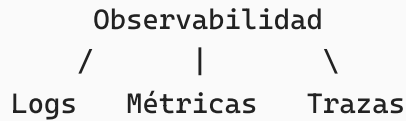
Lo que se necesita:

- Visibilidad completa de cada paso del agente
- Trazabilidad (poder reconstruir exactamente qué pasó en una ejecución)
- Métricas en tiempo real
- Herramientas de debugging
- Evaluación continua (no solo probar una vez, sino medir continuamente)

¿Qué es la Observabilidad?

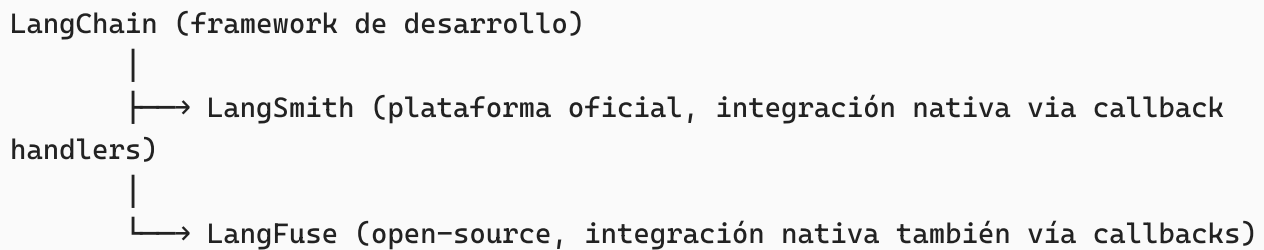
Definición: Capacidad de entender el estado interno de un sistema basándose en los datos que produce.

Se compone de tres pilares:



- **Logs:** registros de eventos puntuales ("el agente llamó a la tool X con estos argumentos")
- **Métricas:** números agregados a lo largo del tiempo (latencia promedio, tasa de error)
- **Trazas (Traces):** la secuencia completa y jerárquica de todos los pasos de una ejecución específica — esto es lo más importante y específico para agentes, porque te permite ver "el árbol de decisiones" completo de una tarea

Ecosistema de Herramientas



Ambas herramientas se "enganchan" al framework mediante **callback handlers** — código que se ejecuta automáticamente en cada paso del agente (cuando empieza, cuando llama una tool, cuando termina) y envía esa información a la plataforma de observabilidad.

10. LangSmith

Visión General

Plataforma oficial de LangChain para desarrollo, monitoreo y mejora continua de aplicaciones LLM.

Características:

- Integración nativa (con LangChain, sin fricción)
- Plataforma en la nube
- UI intuitiva
- Soporte oficial

Modelo de negocio: plan gratuito limitado, planes pagos por uso, soporte empresarial con SLA garantizado.

Funcionalidades Principales

1. Debugging y Trazas

- Visualización jerárquica de llamadas (ves el árbol completo: agente → tool 1 → sub-llamada → tool 2...)
- Inspección de inputs/outputs en cada nodo
- Identificación de cuellos de botella (qué paso tomó más tiempo)

2. Monitoreo en Producción

- Métricas en tiempo real
- Dashboards personalizables
- Alertas automáticas (ej. "avísame si la tasa de error supera 5%")

3. Testing y Evaluación

- Datasets de prueba (conjuntos de inputs con outputs esperados)
- Evaluadores automáticos (otro LLM o reglas que califican si la respuesta es buena)
- Comparación de versiones (¿el cambio de prompt mejoró o empeoró el rendimiento?)

Dataset → Modelo → Evaluación → Resultados
(inputs) (genera) (califica) (reporta)

Métricas Clave en LangSmith

Performance:

- Latencia: P50, P95, P99 por componente (percentiles — P95 significa "el 95% de las requests fueron más rápidas que este valor")
- Throughput: requests por segundo
- Tasa de error: fallos y tipos de fallo

Costos:

- Tokens: input vs output (recuerda del Taller 1 que el output suele ser más caro)
- Costo por request: desglose por modelo usado
- Tendencias: evolución del costo en el tiempo

Integración con LangChain

1. Configuración: variables de entorno + API Key de LangSmith
2. Aplicación: tu código LangChain normal, SIN CAMBIOS necesarios
3. Trazas automáticas: se envían transparentemente a LangSmith
4. Visualización: dashboard web para análisis y debugging

La gran ventaja: si ya usas LangChain, activar LangSmith es prácticamente gratis en esfuerzo de implementación — solo configurar una API key.

11. LangFuse

Visión General

Plataforma open-source de observabilidad para aplicaciones LLM con enfoque en privacidad y control.

Características:

- 100% Open Source
- Self-hosting posible (lo corres en tu propia infraestructura)
- Control total de datos
- Comunidad activa

Modelo de negocio: cloud gratis (limitado), self-hosted gratis, cloud Pro (pago), soporte enterprise.

Funcionalidades Principales

Observabilidad Completa:

- Trazas detalladas multi-nivel
- Métricas agregadas
- Logs estructurados

Gestión de Usuarios:

- Seguimiento por sesión
- Análisis por usuario individual
- User feedback integrado (los usuarios pueden calificar respuestas, y eso se vincula a la traza)

Anotación de Datos:

- Etiquetar interacciones manualmente (ej. marcar cuáles respuestas fueron buenas/malas)
- Crear datasets de entrenamiento a partir de interacciones reales
- Feedback loop cerrado: usas los datos de producción para mejorar el sistema

Integración con LangChain

```
Tu código LangChain (Chains, Agents)
    |
    ▼
CallbackHandler de LangFuse (captura eventos)
    |
    ▼
Plataforma LangFuse (Cloud o self-hosted)
```

Ventaja clave: mínima modificación del código — máxima información capturada.

12. LangSmith vs LangFuse: Comparación

Característica	LangSmith	LangFuse
Integración LangChain	Nativa	Callback
Licencia	Propietaria	Open Source
Hosting	Solo cloud	Cloud + Self-hosted
Costo gratuito	Limitado	Generoso
UI/UX	Excelente	Muy buena
Documentación	Completa	Buena
Soporte oficial	Sí	Comunidad
Extensibilidad	Limitada	Total
Privacy	Depende del plan	Control total
Evaluadores	Incluidos	Configurables
Analytics	Avanzado	Bueno

¿Cuándo Usar Cada Uno?

LangSmith es ideal si:

- Usas LangChain intensivamente

- Quieres setup mínimo
- Priorizas soporte oficial
- Puedes usar cloud (no hay restricciones de datos sensibles)
- Tienes budget para herramientas pagas

LangFuse es ideal si:

- Necesitas datos on-premise (ej. por regulaciones, datos médicos/financieros)
- Quieres personalización total
- Prefieres open source (evitar vendor lock-in)
- Manejas datos muy sensibles
- Tu budget es limitado

Importante: ¡Puedes usar ambos! No son mutuamente excluyentes — son complementarios, no competidores directos en la práctica.

Estrategia Híbrida Recomendada

Desarrollo	→ LangSmith	(debugging rápido, UI intuitiva, feedback inmediato)
Producción	→ LangFuse	(control de datos, sin límites, analytics custom)
Custom	→ Sistema propio	(métricas de negocio muy específicas)

13. Flujos de Trabajo

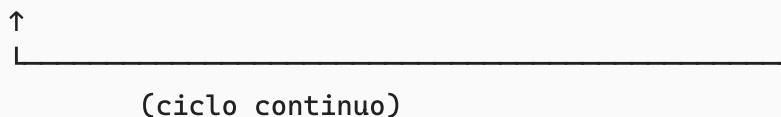
Flujo de Trabajo: Desarrollo

1. Escribir código
2. Probar localmente
3. Ver trazas
4. Identificar issues
5. Corregir
6. Iterar

En esta fase, **LangSmith es ideal** porque: UI intuitiva, feedback rápido, fácil de compartir con el equipo (puedes mandar un link a una traza específica para que un compañero vea exactamente qué pasó).

Flujo de Trabajo: Producción

Deploy a Producción → Monitoreo Continuo → Alertas → Análisis Profundo → Mejora



En producción, **LangFuse aporta ventajas**: datos propios (compliance), APIs completas para integrar con tus propios dashboards, sin límites de volumen, analytics personalizados según métricas de negocio.

14. Mejores Prácticas y Métricas

Métricas Importantes a Seguir

Métricas de Usuario:

- Latencia P95, P99 (la experiencia del "peor caso" importa más que el promedio)
- Tasa de éxito (¿el agente completó la tarea correctamente?)
- User satisfaction score (feedback directo del usuario)

Métricas de Negocio:

- Costo por request
- Requests por usuario (¿estás dentro del presupuesto esperado por usuario activo?)
- ROI de la aplicación (¿el valor generado supera el costo de operar el agente?)

Métricas Técnicas:

- Token usage por componente (¿qué parte del sistema consume más tokens? — ayuda a optimizar)
- Cache hit rate (si cacheas resultados de tools o respuestas del LLM, ¿qué porcentaje se reutiliza?)

Checklist de Mejores Prácticas (síntesis del taller)

1. **Define guardrails desde el diseño**, no como parche después de un incidente
2. **Usa human-in-the-loop** para cualquier acción irreversible o costosa
3. **Combina LLM + lógica determinista**: no todo necesita pasar por el modelo
4. **Limita las iteraciones** en patrones de bucle para evitar costos descontrolados
5. **Instrumenta desde el día 1**: agrega observabilidad antes de tener problemas, no después

- 6. **Elige el patrón según la tarea:** no uses un sistema multi-agente complejo si un solo agente basta
- 7. **Mide costo y latencia constantemente:** un agente que "funciona" pero cuesta 10x lo esperado no es exitoso
- 8. **Versiona tus prompts** y compara resultados al cambiarlos (esto es exactamente lo que ofrece la función de "Testing y Evaluación" de LangSmith)

15. Glosario Rápido

Término	Significado
Agente	LLM + capacidades de razonamiento, acción y memoria autónomas
Orchestrator	Componente que coordina el flujo entre LLM, tools y memoria
Guardrails	Mecanismos de seguridad que limitan acciones del agente
Tool / Function Calling	Función que el LLM puede invocar para actuar en el mundo real
MCP	Protocolo estándar (JSON-RPC 2.0) para conectar LLMs con datos/herramientas
ReAct	Patrón de Razonamiento + Acción en ciclos (Thought → Action → Observation)
Reflexion	Patrón donde el agente critica/refina su propio output
Multi-Agent	Sistema con múltiples agentes especializados colaborando
Human-in-the-loop	Patrón donde un humano aprueba acciones críticas antes de ejecutarse
Observabilidad	Capacidad de entender el estado interno de un sistema (logs + métricas + trazas)
Traza (Trace)	Registro jerárquico completo de una ejecución del agente
LangSmith	Plataforma de observabilidad oficial de LangChain
LangFuse	Plataforma de observabilidad open-source

Preguntas de Revisión

- 1. ¿Cuál es la diferencia fundamental entre un LLM simple y un agente?
- 2. Explica el ciclo ReAct con un ejemplo propio (distinto al de clima).

3. ¿Cuándo usarías un patrón de Coordinador en lugar de un patrón Secuencial?
 4. ¿Por qué MCP es preferible a Function Calling tradicional cuando tienes múltiples agentes y frameworks?
 5. Diseña (en diagrama) un sistema de agentes para automatizar la atención al cliente de una tienda online, identificando qué patrones usarías y por qué.
 6. ¿Por qué la observabilidad es más crítica en sistemas de agentes que en aplicaciones de software tradicional?
 7. Si tu empresa maneja datos médicos sensibles, ¿LangSmith o LangFuse? Justifica.
 8. ¿Qué guardrail añadirías a un agente que tiene acceso a una tool de "enviar_email"?
-

Referencias y Lecturas Recomendadas

Paper Fundamental

- Yao et al. (2022) — "ReAct: Synergizing Reasoning and Acting in Language Models"
- Shinn et al. (2023) — "Reflexion: Language Agents with Verbal Reinforcement Learning"

Documentación Oficial

- [LangChain Docs](#)
 - [LangGraph Docs](#)
 - [Model Context Protocol — Spec oficial](#)
 - [LangSmith](#)
 - [LangFuse](#)
 - [CrewAI](#)
 - [AutoGen \(Microsoft\)](#)
-

Dr. Vicente Machaca Arceda — `vmachaca@utec.edu.pe`

Este documento integra los contenidos del Taller 2 con expansiones conceptuales y ejemplos de código para construir una comprensión sólida y aplicable del diseño de agentes con LLMs.